

COMP 2402

Abstract Data Types & Algorithms

Welcome to COMP 2402

Alina Shaikhet – Instructor

Email: alina.shaikhet@carleton.ca

Office Hours: Tuesday & Thursday 16:00 – 17:00

Course expectations

What will you learn and why?

(code that is efficient, reliable, fast, and elegant)



You will learn how to write **better code** leading to software that runs **faster** and consumes **less memory**.

You will be able to weigh the pros and cons of various solutions to the given problem and be able to make educated decisions as to which code is best for the given situation.

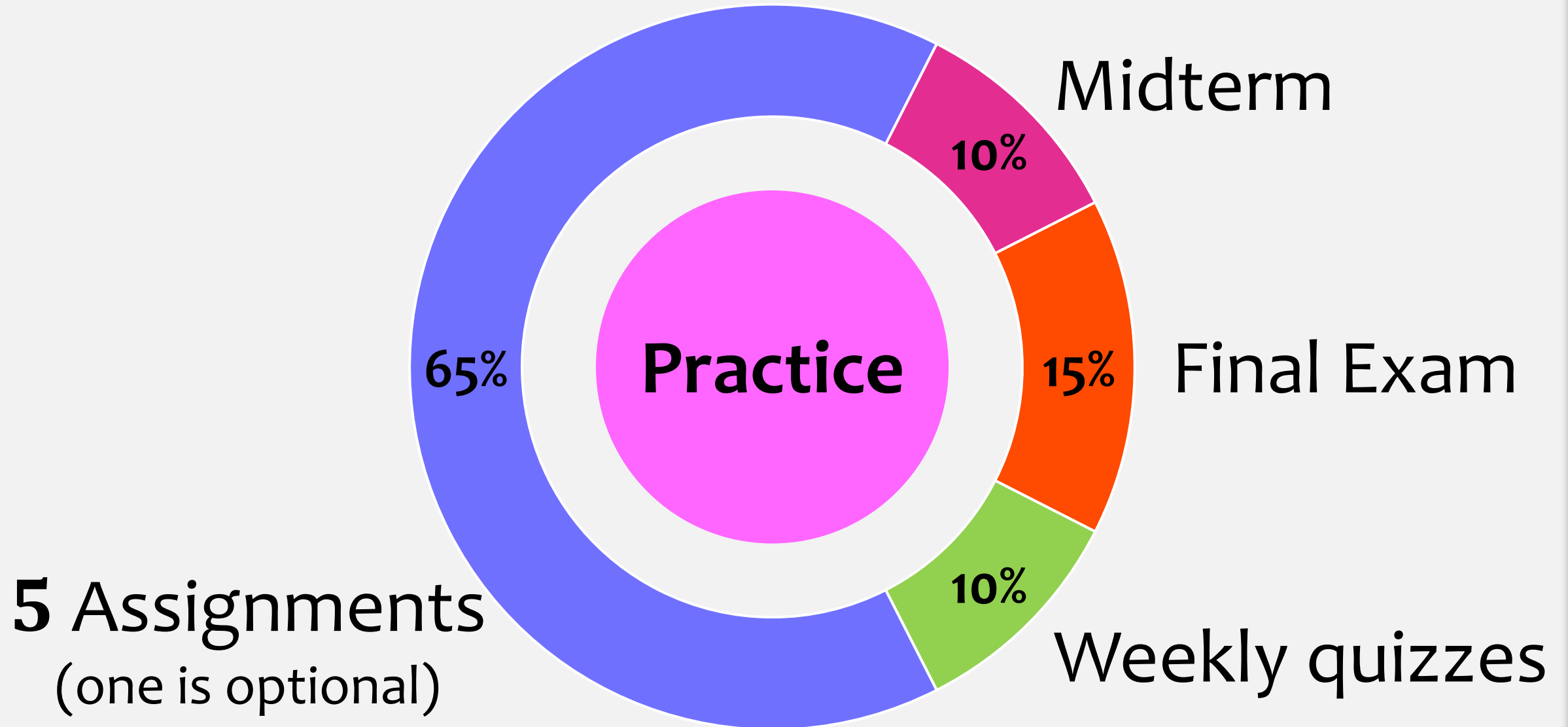
This course is fundamental to computer science. When you are applying for software developing (or engineering) positions, I guarantee you will be tested on problems that involve the use of data structures and algorithms.

Topics

This course is about how to use, implement, and analyze data structures.

- The Java Collections Framework (JCF)
- Sequences: lists, stacks, queues, deques
- Array-based implementations of sequences
- Linked-list based implementations of sequences
- Unordered sets - hash tables
- Ordered sets - balanced search trees, skiplists
- Priority queues - heaps
- Sorting algorithms
- Graphs
- **Performance issues**

Evaluation



What you will need for the course

Background:

- Review your Java programming
- Review discrete math (in particular, big-Oh notation and Sigma-notation for sums).
- Textbook (it is free). It was specifically written for this course.
- Install Java (command line compiler)
If you do not have Java installed on your computer, you can download it for free at [oracle.com](https://www.oracle.com)
- Java Tutorials (optional) <https://www.w3schools.com/java/default.asp>
- Editor. The most popular choices are [Visual Studio Code](#) and [Atom](#).

Assignments

Assignments are graded by **automatic submission server**.

- Instant feedback
- Submit often – your best grade is recorded
- TA time is allocated to helping you.



- No marks for trying.
- If your submission is improperly packaged, or your code is not compiling for whatever reason, you will get a mark of **0**.

Assignment Workshops

Mondays 11:00 – 12:00 – Assignment Workshop by Justin Connolly

Tuesdays 17:00 – 18:00 – Assignment Workshop by Aylin Eрман

Workshops will be conducted via Discord.

Workshops will not be happening every week. Refer to the calendar for details.

Assignments

No need to submit assignments on Brightspace.

- Start early and ensure you have significant time to work on your assignments – this will allow you to learn the concepts better
- Do not be late. The server cuts off – that's it. I post solutions shortly after the deadline. No late submissions can be accepted after that.

but you do get to drop the lowest assignment

- Collaboration/discussions are encouraged, but at a high level (general approach, useful tools, helpful resources)
- Do not share any of your work with others, do not copy someone else's work from any source (students, lectures, TAs, online, etc.)
Plagiarism is a form of theft.

How to get good at this class

Our goal is not only to solve a problem, but to do it in the best way possible.

- Take notes
- Practice

A rewarding aspect of programming is that once you have done a program correctly, you get to see it work.
- Discuss with other students, make friends
- Ask questions

My goal #1 is to create a positive learning environment for everybody.

I want you to succeed to the best of your ability. If there's something I can do to help you learn, please let me know.
- Support each other

By the end of this course, you will be a better problem solver!

Abstract data types and algorithms

We will learn:

- how to be more careful and thoughtful about the way you program,
- how to store and manipulate data better, and how to choose the best way to do that for the situation that you're in.

You want programs that run

- Correctly
- Fast
- Reliably
- On huge inputs
- Space efficiently

3 Main Components

- **Data structures** is a collection of:
 - data values,
 - relationships among them,
 - functions/operations that can be applied to the data.
 - **Algorithms**
 - 1. Terminate?
 - 2. Correct?
 - 3. Efficient?
 - 1. Time complexity
 - 2. Space complexity
- An **algorithm** is a finite sequence of precise instructions (or steps) for solving a problem
- **Abstraction** – hiding implementation details in order to reduce complexity and increase manageability

Data Structures

- What are they?

Data Structures – store data,
answer queries,
(maybe) perform updates.

numbers, strings, documents, ...

what is the data stored at position i ? – `get(i)`
what is the “smallest” item? – `first()`

add/remove/change data
add the element x at position i – `add(i,x)`

- Why do we need them?

Every part of a computer system has non-trivial data structures in it.

We interact with
data structures
all the time:

- Open a file
- Look up a contact on your phone
- Web search
- Log in
- GIS (geographic systems)



In this Course we will

- Define a new abstract data types (ADT), i.e an **interface** that provides methods that allow you to manipulate data in certain ways
- Discuss various **implementations** of the ADTs using different structures and algorithms
- Discuss pros/cons.

Interface

What does the data structure represent?

A collection, a set, a sequence,
a map, a graph, the world, ...

What operations does it support?

adding, removing, finding elements;
membership testing;
range searching; ...

Implementation

Performance: speed and memory

How long does each operation take?

How much space does it use?

example

Sets (or sorted sets)
Sequences
Maps
Graphs

Binary search trees
Skiplists
Hash tables
Adjacency Lists

You can have different implementations of the same interface

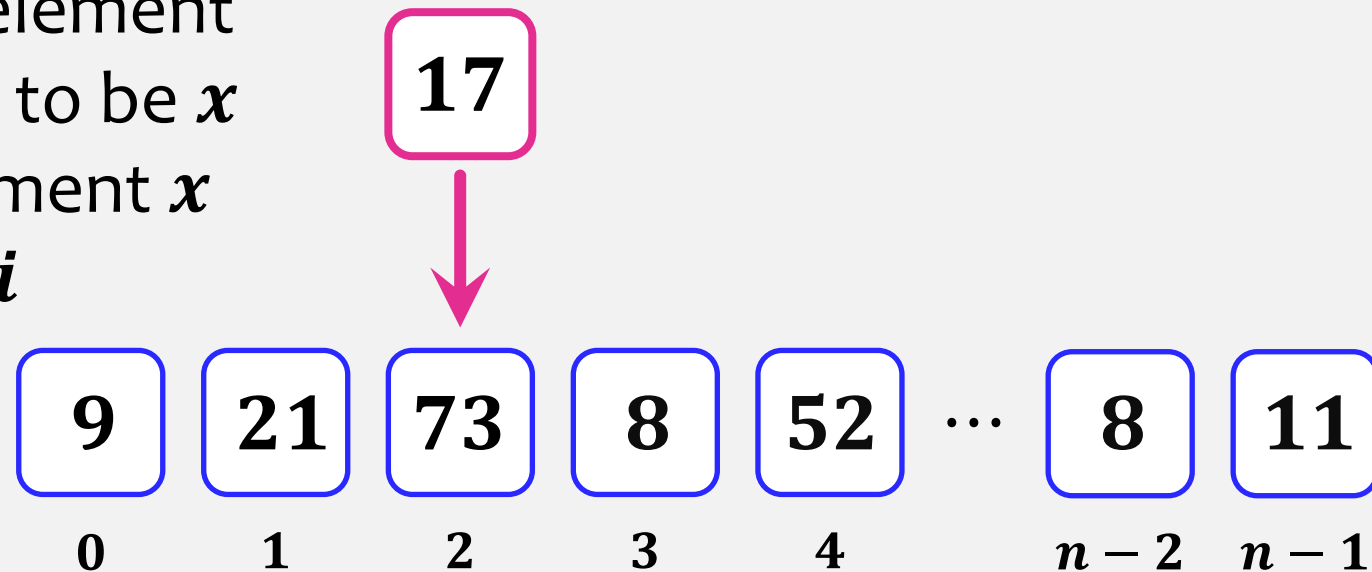
Abstract data types – List

Lists – store an indexed sequence of elements

Example: movie
list(season 1,2 3,...)

Methods:

- `size()` – returns the number of elements on the list (n)
- `get( $i$ )` – returns the element at position i
- `set( $i, x$ )` – update the element at position i to be x
- `add( $i, x$ )` – add the element x to position i



Abstract data types – List

Lists – store an indexed sequence of elements

Example: movie
list(season 1,2 3,...)

Methods:

- `size()` – returns the number of elements on the list (n)
- `get( $i$ )` – returns the element at position i
- `set( $i, x$ )` – update the element at position i to be x
- `add( $i, x$ )` – add the element x to position i
- `remove( $i$ )` – remove element at position i

